

# ESSE v1.0

## Technical Blueprint & Coding Guide

Panduan Implementasi untuk AI Coding Agent (Rust Ecosystem)

|                 |                                        |
|-----------------|----------------------------------------|
| Versi Dokumen   | 1.0                                    |
| Target Compiler | rustc 1.78+ (stable)                   |
| Paradigma       | Async-first · Zero-copy · Memory-safe  |
| Dibuat Untuk    | OpenAI Codex / Cursor / GitHub Copilot |

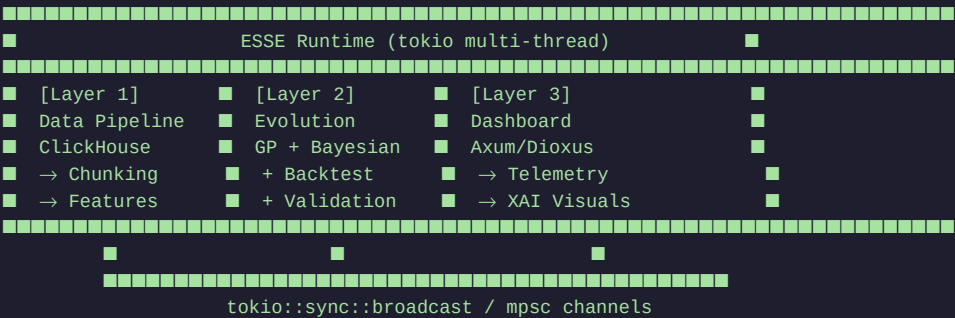
# Panduan Implementasi untuk AI Coding Agent (Rust Ecosystem)

Versi Dokumen: 1.0 | Target Compiler: rustc 1.78+ (stable) | Paradigma: Async-first, Zero-copy, Memory-safe

## BAGIAN 1: Project Setup & Architecture Overview

### 1.1 Arsitektur Tingkat Tinggi

ESSE adalah sistem **multi-layer concurrent pipeline** yang beroperasi penuh secara asinkron. Tiga lapisan utama berjalan secara paralel sebagai tokio tasks:



#### Prinsip desain:

- Semua I/O (ClickHouse, PostgreSQL, HTTP) adalah **async non-blocking** via **tokio**.
- CPU-bound work (evaluasi AST, feature computation) didelegasikan ke **rayon thread pool** melalui **tokio::task::spawn\_blocking**.
- GPU path (opsional) menggunakan **cudarc / wgpu** untuk SIMT batch evaluation.
- Tidak ada global mutable state — semua shared state dibungkus dalam **Arc** atau dikomunikasikan via channels.

### 1.2 Project Tree Structure

```
esse/
├── Cargo.toml
├── Cargo.lock
├── config/
│   ├── default.toml           # Konfigurasi runtime (DB URLs, hyperparams)
│   └── python/
│       └── optuna_bridge.py    # Script Python yang dipanggil via PyO3
├── migrations/
│   └── 001_initial_schema.sql  # PostgreSQL DDL (strategy_dna, trial_logs, dll)
├── src/
│   ├── main.rs                # Entry point: boot tokio runtime, spawn 3 layer tasks
│   ├── config.rs              # Struct Config, parsing TOML via `config` crate
│   ├── error.rs               # Custom error enum (EsseError) via `thiserror`
│   └── data/                  # [FASE 1] Data Layer
│       ├── mod.rs
│       ├── types.rs           # Struct Tick, Candle, Instrument enum, Timeframe enum
│       ├── clickhouse.rs      # Client pool, query builder, gapless chunking
│       ├── postgres.rs        # sqlx Pool, CRUD untuk strategy_dna, trial_logs, dll
│       └── context.rs          # Struct ChunkContext – stateful hand-off antar chunk
│   └── features/              # [FASE 2] Feature Engineering (9 Microstructure)
│       ├── mod.rs
│       ├── pipeline.rs        # Orchestrator: input Tick slice → output FeatureRow
│       ├── obi.rs             # Order Book Imbalance
│       ├── tick_velocity.rs   # Mid-price velocity
│       ├── spread.rs          # Spread Dynamics & Volume Weighted Spread
│       └── density.rs         # Tick Density
```

```

■ ■ ■■ volume_skew.rs      # Volume Clock Skew
■ ■ ■■ momentum.rs        # Mid Price Momentum (dual-window)
■ ■ ■■ higher_moments.rs   # Rolling Skewness & Kurtosis
■ ■ ■■ hurst.rs            # Rolling Hurst Exponent (R/S Analysis)
■ ■
■ ■■ gp/                  # [FASE 3] Genetic Programming Engine
■ ■ ■■ mod.rs
■ ■ ■■ ast.rs              # Enum AstNode, evaluator, depth/node counter
■ ■ ■■ population.rs       # Struct Individual, Population, init/selection
■ ■ ■■ operators.rs        # Crossover, mutation, bloat control
■ ■ ■■ diversity.rs        # Genotypic (Tree Edit Distance), Phenotypic distance
■ ■ ■■ bayesian.rs         # Py03 FFI bridge ke Optuna (TPE)
■ ■
■ ■■ backtest/            # [FASE 4] Backtest & Validation
■ ■ ■■ mod.rs
■ ■ ■■ engine.rs           # Core tick-level simulation loop
■ ■ ■■ position.rs        # Struct Position, execution guardrails
■ ■ ■■ metrics.rs          # ProfitFactor, UlcerIndex, DrawdownStats, dll
■ ■ ■■ walk_forward.rs     # Multi-window IS/OOS split matrix generator
■ ■ ■■ regime.rs           # Hurst-based + ATR regime classifier
■ ■ ■■ noise_injection.rs  # Slippage/spread perturbation untuk stress test
■ ■ ■■ filter.rs           # Hard filters + OOS Degradation Rule
■ ■
■ ■■ evolution/           # [FASE 3+4] Orchestration Loop
■ ■ ■■ mod.rs
■ ■ ■■ fitness.rs          # Fungsi Fitness Score (formula weighted)
■ ■ ■■ orchestrator.rs     # Generational loop, Top-N selection, export JSON
■ ■
■ ■■ dashboard/           # [FASE 5] Observability
■ ■ ■■ mod.rs
■ ■ ■■ server.rs           # Axum HTTP server setup, routing
■ ■ ■■ telemetry.rs        # Struct TelemetryState (Arc<RwLock>), broadcast channel
■ ■ ■■ api/
■ ■ ■■ evolution.rs        # GET /api/evolution/overview
■ ■ ■■ strategy.rs         # GET /api/strategy/:id/deep-dive
■ ■ ■■ regime.rs           # GET /api/regime/heatmap
■ ■ ■■ system.rs           # GET /api/system/telemetry
■ ■ ■■ static/             # Frontend assets (Dioxus WASM atau HTML/JS)
■
■■■ tests/
■■■ integration/
■ ■■ data_pipeline_test.rs
■ ■■ backtest_determinism_test.rs
■■■ unit/
■ ■■ ast_eval_test.rs
■ ■■ feature_correctness_test.rs

```

## BAGIAN 2: Dependency Management (Cargo.toml)

```

[package]
name = "esse"
version = "0.1.0"
edition = "2021"
rust-version = "1.78"

[[bin]]
name = "esse"
path = "src/main.rs"

[features]
default = ["cpu"]
cpu = []
gpu-cuda = ["cudarc"]
gpu-wgpu = ["dep:wgpu"]

[dependencies]

# === ASYNC RUNTIME ===

```

```

tokio = { version = "1", features = ["full", "tracing"] }

# === DATA LAYER ===
clickhouse = { version = "0.12", features = ["tls", "lz4"] } # clickhouse.rs
sqlx = { version = "0.8", features = [
    "runtime-tokio-rustls",
    "postgres",
    "chrono",
    "uuid",
    "json",
    "migrate"
] }

# === SERIALIZATION ===
serde = { version = "1", features = ["derive"] }
serde_json = "1"
uuid = { version = "1", features = ["v4", "serde"] }
chrono = { version = "0.4", features = ["serde"] }

# === ANALYTICS / DATAFRAME ===
polars = { version = "0.41", features = [
    "lazy",
    "streaming",
    "rolling_window",
    "moment",      # skewness, kurtosis
    "describe",
    "dtype-decimal"
] }

# === PARALLELISM (CPU) ===
rayon = "1.10"

# === PYTHON FFI (BAYESIAN OPTUNA) ===
pyo3 = { version = "0.22", features = ["auto-initialize", "gil-refs"] }

# === GPU ACCELERATION (feature-gated) ===
cudarc = { version = "0.12", features = ["cuda-12050", "driver", "nVRTC"], optional = true }
wgpu = { version = "22", optional = true }

# === WEB FRAMEWORK (DASHBOARD) ===
axum = { version = "0.7", features = ["ws", "multipart"] }
tower = { version = "0.5", features = ["full"] }
tower-http = { version = "0.6", features = ["cors", "fs", "trace"] }

# === ERROR HANDLING ===
thiserror = "1"
anyhow = "1"

# === LOGGING & TRACING ===
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter", "json"] }

# === CONFIG ===
config = { version = "0.14", features = ["toml"] }

# === MATH / NUMERICS ===
statrs = "0.17"                # Distribusi statistik
ordered-float = "4"            # OrderedFloat untuk sorting fitness
rust_decimal = { version = "1", features = ["serde-with-str"] }
num-traits = "0.2"

# === TREE EDIT DISTANCE (GP Diversity) ===
# Implementasi manual – lihat src/gp/diversity.rs
# Alternatif: gunakan `zss` crate jika tersedia

# === UTILITIES ===
itertools = "0.13"
dashmap = "6"                  # Concurrent HashMap untuk populasi cache
parking_lot = "0.12"          # Faster Mutex/RwLock
bytes = "1"
futures = "0.3"

```

```

pin-project-lite = "0.2"
tokio-stream = "0.1"

[dev-dependencies]
criterion = { version = "0.5", features = ["html_reports"] }
proptest = "1"
tokio-test = "0.4"
approx = "0.5"

[profile.release]
opt-level = 3
lto = "fat"
codegen-units = 1
panic = "abort"

[profile.bench]
inherits = "release"
debug = true

```

## BAGIAN 3: Core Data Structures & Type Definitions

### 3.1 Primitif Market Data (`src/data/types.rs`)

```

use chrono::{DateTime, Utc};
use rust_decimal::Decimal;
use serde::{Deserialize, Serialize};

/// 16 instrumen FX yang didukung – harus match persis dengan ClickHouse Enum8
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
#[repr(u8)]
pub enum Instrument {
    XAUUSD = 1, AUDJPY = 2, CHFJPY = 3, CADJPY = 4, GBPJPY = 5,
    EURJPY = 6, NZDJPY = 7, USDJPY = 8, GBPUSD = 9, EURUSD = 10,
    USDCHF = 11, USDCAD = 12, AUDUSD = 13, NZDUSD = 14, EURGBP = 15,
    EURNZD = 16,
}

/// Timeframe resolusi agregat – match dengan ClickHouse Enum8
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
#[repr(u8)]
pub enum Timeframe {
    M1=1, M2=2, M3=3, M4=4, M5=5, M6=6, M10=7, M12=8, M15=9,
    M20=10, M30=11, H1=12, H2=13, H3=14, H4=15, H6=16, H8=17, H12=18,
    D1=19, W1=20,
}

/// Raw tick – deserialized langsung dari ClickHouse row
/// Gunakan f64 internal untuk komputasi kecepatan; convert dari Decimal saat load.
#[derive(Debug, Clone)]
pub struct Tick {
    pub timestamp: DateTime<Utc>,
    pub instrument: Instrument,
    pub bid: f64,
    pub ask: f64,
    pub bid_volume: u32,
    pub ask_volume: u32,
}

impl Tick {
    #[inline(always)]
    pub fn mid(&self) -> f64 { (self.bid + self.ask) * 0.5 }

    #[inline(always)]
    pub fn spread(&self) -> f64 { self.ask - self.bid }
}

/// Candle agregat – digunakan untuk Regime Classification

```

```
#[derive(Debug, Clone)]
pub struct Candle {
    pub timestamp: DateTime<Utc>,
    pub instrument: Instrument,
    pub timeframe: Timeframe,
    pub open: f64,
    pub high: f64,
    pub low: f64,
    pub close: f64,
    pub tick_count: u32,
    pub min_spread: f64,
    pub max_spread: f64,
    pub avg_spread: f64,
    pub total_bid_volume: u64,
    pub total_ask_volume: u64,
    pub vwap: f64,
}
```

### 3.2 Feature Vector (`src/features/mod.rs`)

```
/// Output dari Feature Pipeline – satu row per tick yang telah diproses.
/// Semua field adalah f64 untuk kompatibilitas dengan AST evaluator.
#[derive(Debug, Clone, Default)]
pub struct FeatureRow {
    pub timestamp_ns: i64,           // Unix nanoseconds untuk sorting
    pub obi: f64,                    // Order Book Imbalance [-1.0, 1.0]
    pub tick_velocity: f64,          //  $\Delta \text{mid} / \Delta t$  (pips/sec)
    pub spread_dynamics: f64,        // Rolling std-dev of spread
    pub tick_density: f64,           // ticks per second dalam window
    pub volume_clock_skew: f64,      //  $\text{bid\_vol} / (\text{bid\_vol} + \text{ask\_vol}) - 0.5$ 
    pub mid_momentum: f64,           //  $\text{momentum\_short} - \text{momentum\_long}$ 
    pub rolling_skewness: f64,       // Pearson's moment coefficient of skewness
    pub rolling_kurtosis: f64,       // Excess kurtosis
    pub volume_weighted_spread: f64, //  $\Sigma(\text{spread} * \text{volume}) / \Sigma(\text{volume})$ 
    pub hurst_exponent: f64,         // R/S Analysis result [0.0, 1.0]
}

/// Window config untuk rolling computation
#[derive(Debug, Clone, Copy)]
pub struct WindowConfig {
    pub short_window: usize, // Tick count: 3-300
    pub long_window: usize,  // Untuk dual-window momentum
    pub hurst_window: usize, // Minimal 50 ticks direkomendasikan
}
```

### 3.3 Abstract Syntax Tree / GP (`src/gp/ast.rs`)

```
use std::fmt;

/// Terminal nodes – leaf dari pohon GP, merujuk ke indeks FeatureRow
#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
pub enum Terminal {
    Obi,
    TickVelocity,
    SpreadDynamics,
    TickDensity,
    VolumeClockSkew,
    MidMomentum,
    RollingSkewness,
    RollingKurtosis,
    VolumeWeightedSpread,
    HurstExponent,
    Constant(f64), // Evolusi konstanta numerik
}

/// Fungsi primitif – operator matematika pada node AST
```

```

#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
pub enum Primitive {
    Add, Sub, Mul, Div,
    Max2, Min2,
    Neg, Square, Cube,
    Log, Sqrt, Sigmoid, Sign,
}

/// Node utama AST – tree recursive menggunakan Box untuk heap allocation
#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
pub enum AstNode {
    Leaf(Terminal),
    /// Unary: 1 child
    Unary { op: Primitive, child: Box<AstNode> },
    /// Binary: 2 children
    Binary { op: Primitive, left: Box<AstNode>, right: Box<AstNode> },
}

/// Konstanta batas bloat – HARD LIMIT, jangan ubah tanpa pertimbangan matang
pub const MAX_DEPTH: usize = 6;
pub const MAX_NODES: usize = 60;
pub const MIN_NODES: usize = 2;

impl AstNode {
    /// Evaluasi AST terhadap satu FeatureRow – MUST be pure/no side effects
    pub fn evaluate(&self, features: &crate::features::FeatureRow) -> f64 {
        match self {
            AstNode::Leaf(terminal) => terminal.evaluate(features),
            AstNode::Unary { op, child } => {
                let v = child.evaluate(features);
                op.apply_unary(v)
            }
            AstNode::Binary { op, left, right } => {
                let l = left.evaluate(features);
                let r = right.evaluate(features);
                op.apply_binary(l, r)
            }
        }
    }

    /// Hitung jumlah total node secara rekursif
    pub fn node_count(&self) -> usize {
        match self {
            AstNode::Leaf(_) => 1,
            AstNode::Unary { child, .. } => 1 + child.node_count(),
            AstNode::Binary { left, right, .. } => 1 + left.node_count() + right.node_count(),
        }
    }

    /// Hitung kedalaman pohon secara rekursif
    pub fn depth(&self) -> usize {
        match self {
            AstNode::Leaf(_) => 0,
            AstNode::Unary { child, .. } => 1 + child.depth(),
            AstNode::Binary { left, right, .. } => 1 + left.depth().max(right.depth()),
        }
    }

    /// Validasi bloat constraints – panggil setelah setiap mutasi/crossover
    pub fn is_valid(&self) -> bool {
        let nc = self.node_count();
        let d = self.depth();
        nc >= MIN_NODES && nc <= MAX_NODES && d <= MAX_DEPTH
    }
}

impl Primitive {
    pub fn apply_unary(&self, v: f64) -> f64 {
        match self {
            Primitive::Neg => -v,
            Primitive::Square => v * v,
        }
    }
}

```

```

        Primitive::Cube => v * v * v,
        Primitive::Log => if v > 0.0 { v.ln() } else { f64::NEG_INFINITY },
        Primitive::Sqrt => if v >= 0.0 { v.sqrt() } else { 0.0 },
        Primitive::Sigmoid => 1.0 / (1.0 + (-v).exp()),
        Primitive::Sign => v.signum(),
        _ => unreachable!("Non-unary op called as unary"),
    }
}

pub fn apply_binary(&self, l: f64, r: f64) -> f64 {
    match self {
        Primitive::Add => l + r,
        Primitive::Sub => l - r,
        Primitive::Mul => l * r,
        Primitive::Div => if r.abs() > 1e-10 { l / r } else { 0.0 }, // Protected division
        Primitive::Max2 => l.max(r),
        Primitive::Min2 => l.min(r),
        _ => unreachable!("Non-binary op called as binary"),
    }
}

impl Terminal {
    pub fn evaluate(&self, f: &crate::features::FeatureRow) -> f64 {
        match self {
            Terminal::Obi => f.obi,
            Terminal::TickVelocity => f.tick_velocity,
            Terminal::SpreadDynamics => f.spread_dynamics,
            Terminal::TickDensity => f.tick_density,
            Terminal::VolumeClockSkew => f.volume_clock_skew,
            Terminal::MidMomentum => f.mid_momentum,
            Terminal::RollingSkewness => f.rolling_skewness,
            Terminal::RollingKurtosis => f.rolling_kurtosis,
            Terminal::VolumeWeightedSpread => f.volume_weighted_spread,
            Terminal::HurstExponent => f.hurst_exponent,
            Terminal::Constant(c) => *c,
        }
    }
}

```

### 3.4 Individual & Population (`src/gp/population.rs`)

```

use uuid::Uuid;
use serde::{Deserialize, Serialize};

/// Representasi satu individu dalam populasi GP
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Individual {
    pub id: Uuid,
    pub ast: AstNode,
    pub params: StrategyParams,
    pub fitness: Option<f64>, // None jika belum dievaluasi
    pub oos_metrics: Option<OosMetrics>,
    pub regime_profile: Option<RegimeProfile>,
    pub generation_born: u32,
}

/// Parameter numerik yang dioptimasi via Bayesian (Optuna)
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct StrategyParams {
    pub sl_pips: f64, // [10.0, 50.0]
    pub tp_pips: f64, // [sl_pips * 1.5, 150.0]
    pub signal_threshold: f64, // [0.05, 0.95]
    pub feature_window: usize, // [3, 300] ticks
}

/// Metrik agregat OOS dari Walk-Forward
#[derive(Debug, Clone, Serialize, Deserialize)]

```



```

pub struct OosMetrics {
    pub avg_profit_factor: f64,
    pub regime_stability_score: f64,
    pub avg_drawdown_pips: f64,
    pub windows_passed: usize,
    pub windows_total: usize,
}

/// Label rezim pasar per jendela OOS
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
pub enum RegimeLabel {
    HighVolTrending,
    LowVolTrending,
    HighVolRanging,
    LowVolRanging,
    RandomNoise,
}

/// Profil ketahanan strategi lintas rezim
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct RegimeProfile {
    pub target_regime: RegimeLabel,
    pub survival_map: std::collections::HashMap<RegimeLabel, f64>, // regime → win_rate
}

```

### 3.5 Fitness Score Formula (`src/evolution/fitness.rs`)

```

/// Komponen fitness sesuai formula spesifikasi:
/// Score = (0.40 * avg_oos_pf) + (0.25 * regime_stability)
///         - (0.20 * drawdown_penalty) + (0.15 * is_avg_perf)
#[derive(Debug, Clone)]
pub struct FitnessComponents {
    pub avg_oos_profit_factor: f64, // Bobot 0.40
    pub oos_regime_stability: f64, // Bobot 0.25
    pub risk_drawdown_penalty: f64, // Bobot 0.20 (dikurangi)
    pub is_average_performance: f64, // Bobot 0.15
}

impl FitnessComponents {
    pub fn compute_score(&self) -> f64 {
        (0.40 * self.avg_oos_profit_factor)
        + (0.25 * self.oos_regime_stability)
        - (0.20 * self.risk_drawdown_penalty)
        + (0.15 * self.is_average_performance)
    }
}

/// Ulcer Index = sqrt( Σ(drawdown_pct²) / n )
pub fn ulcer_index(equity_curve: &[f64]) -> f64 {
    if equity_curve.is_empty() { return 0.0; }
    let peak = equity_curve.iter().cloned().fold(f64::NEG_INFINITY, f64::max);
    let sum_sq: f64 = equity_curve.iter()
        .map(|&e| {
            let dd = (peak - e) / peak.abs().max(1e-10) * 100.0;
            dd * dd
        })
        .sum();
    (sum_sq / equity_curve.len() as f64).sqrt()
}

```

### 3.6 Shared Telemetry State (`src/dashboard/telemetry.rs`)

```

use std::sync::Arc;
use parking_lot::RwLock;
use serde::Serialize;

```

```

/// State yang di-share antara Evolution Engine dan Dashboard via Arc<RwLock>
#[derive(Debug, Default, Serialize, Clone)]
pub struct TelemetryState {
    pub current_generation: u32,
    pub population_size: usize,
    pub best_fitness: f64,
    pub avg_fitness: f64,
    pub diversity_score: f64,          // 0.0 (inbred) → 1.0 (healthy)
    pub elimination_funnel: EliminationFunnel,
    pub ram_usage_mb: f64,
    pub gpu_vram_usage_mb: f64,
    pub rows_per_second: f64,
    pub top_features: Vec<(String, f64)>, // feature_name → dominance_ratio
}

#[derive(Debug, Default, Serialize, Clone)]
pub struct EliminationFunnel {
    pub generated: u64,
    pub failed_hard_filter: u64,
    pub failed_oos: u64,
    pub failed_degradation: u64,
    pub golden_set_count: usize,
}

pub type SharedTelemetry = Arc<RwLock<TelemetryState>>;

```

## BAGIAN 4: Module-by-Module Implementation Guide

### FASE 1: Data Layer

#### 1A. ClickHouse Client & Gapless Chunking (`src/data/clickhouse.rs`)

**Tujuan:** Stream tick data per bulan dengan overlapping warm-up window, tanpa OOM.

```

// Pseudocode implementasi – AI Coder wajib translate ke sintaks penuh

use clickhouse::{Client, Row};

pub struct ClickHouseDataSource {
    client: Client,
    warmup_rows: usize,    // N rows overlapping dari bulan sebelumnya
}

impl ClickHouseDataSource {
    /// Konfigurasi client – gunakan connection pool via Arc<Client>
    pub fn new(url: &str, database: &str) -> Self {
        let client = Client::default()
            .with_url(url)
            .with_database(database)
            .with_option("enable_http_compression", "1");
        Self { client, warmup_rows: 5000 }
    }

    /// Query chunk per bulan dengan overlapping warm-up.
    /// PENTING: ORDER BY (instrument, timestamp) agar deterministik.
    pub async fn stream_ticks_for_month(
        &self,
        instrument: Instrument,
        year: i32,
        month: u32,
    ) -> impl Stream<Item = Result<Tick>> {
        // Hitung batas waktu:
        // start = first day of month - warmup_duration
        // end   = last day of month
        //
        // Query:

```

```

        // SELECT timestamp, bid, ask, bid_volume, ask_volume
        // FROM geonera.ticks
        // WHERE instrument = ? AND timestamp >= ? AND timestamp < ?
        // ORDER BY timestamp
        //
        // Stream menggunakan client.query(...).fetch::<TickRow>()
        // Map setiap row ke struct Tick internal
        todo!("Implementasi streaming query dengan clickhouse.rs")
    }
}

/// Context stateful yang di-hand-off antar chunk
pub struct ChunkContext {
    pub last_ticks: Vec<Tick>,          // N tick terakhir dari chunk sebelumnya
    pub feature_state: FeatureState,    // Rolling state untuk komputasi fitur
    pub strategy_states: Vec<StrategyState>, // State posisi per strategi
}

```

#### Instruksi untuk AI Coder:

1. Gunakan `clickhouse::query::Query::fetch_all()` untuk batch kecil, atau `fetch::()` yang streaming untuk data besar.
2. Implementasikan `TickRow` sebagai struct terpisah yang derive `clickhouse::Row` lalu convert ke `Tick`.
3. Warmup window wajib di-query sebagai bagian dari SQL `WHERE timestamp >= (month_start - warmup_offset)` — bukan fetch terpisah.
4. `ChunkContext` harus `Send + Sync` agar bisa dipindah antar task.

## 1B. PostgreSQL Persistence (`src/data/postgres.rs`)

#### Instruksi:

- Gunakan `sqlx::PgPool` dengan `max_connections: 10`.
- Semua query wajib menggunakan **compile-time checked macros** (`sqlx::query!` / `sqlx::query_as!`).
- `strategy_dna` table menyimpan `ast` sebagai `JSONB` PostgreSQL.
- Gunakan `sqlx migrate run` untuk menjalankan file di `migrations/`.

#### Skema DDL minimal:

```

-- migrations/001_initial_schema.sql

CREATE TABLE IF NOT EXISTS strategy_dna (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ast JSONB NOT NULL,
    params JSONB NOT NULL,
    fitness_score DOUBLE PRECISION,
    status VARCHAR(50) DEFAULT 'candidate',
    oos_profit_factor DOUBLE PRECISION,
    regime_profile JSONB,
    generation_born INTEGER,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS trial_logs (
    id BIGSERIAL PRIMARY KEY,
    strategy_id UUID REFERENCES strategy_dna(id),
    window_id INTEGER,
    regime_id VARCHAR(50),
    profit_factor DOUBLE PRECISION,
    total_pips DOUBLE PRECISION,
    trade_count INTEGER,
    is_oos BOOLEAN,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS trade_logs (

```

```

    id BIGSERIAL PRIMARY KEY,
    strategy_id UUID REFERENCES strategy_dna(id),
    entry_time TIMESTAMPTZ,
    exit_time TIMESTAMPTZ,
    direction VARCHAR(10),
    gross_pips DOUBLE PRECISION,
    net_pips DOUBLE PRECISION,
    window_type VARCHAR(5),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_strategy_dna_fitness ON strategy_dna(fitness_score DESC);
CREATE INDEX idx_trial_logs_strategy ON trial_logs(strategy_id);

```

---

## FASE 2: Feature Engineering

### 2A. Hurst Exponent (`src/features/hurst.rs`)

Hurst Exponent menggunakan **R/S Analysis** (Rescaled Range):

```

/// Hitung Rolling Hurst Exponent via R/S Analysis
/// Input: slice of mid-prices (paling sedikit `window` elemen)
/// Output: H ∈ [0.0, 1.0]
/// H > 0.55 → Trending | H < 0.45 → Mean-Reverting | ~0.5 → Random
pub fn compute_hurst(prices: &[f64]) -> f64 {
    let n = prices.len();
    if n < 20 { return 0.5; } // Insufficient data, return random noise

    // Hitung returns: r_t = price[t] - price[t-1]
    let returns: Vec<f64> = prices.windows(2)
        .map(|w| w[1] - w[0])
        .collect();

    // R/S pada beberapa sub-period sizes menggunakan regresi log-log
    let sizes: Vec<usize> = (2..=n/2)
        .filter(|&s| s.is_power_of_two())
        .collect();

    if sizes.len() < 2 { return 0.5; }

    let mut log_sizes = Vec::new();
    let mut log_rs = Vec::new();

    for size in &sizes {
        if let Some(rs) = rescaled_range(&returns[..returns.len().min(*size)]) {
            log_sizes.push((*size as f64).ln());
            log_rs.push(rs.ln());
        }
    }

    if log_sizes.len() < 2 { return 0.5; }

    // OLS slope = Hurst Exponent
    ols_slope(&log_sizes, &log_rs).clamp(0.0, 1.0)
}

fn rescaled_range(data: &[f64]) -> Option<f64> {
    let n = data.len();
    if n == 0 { return None; }
    let mean = data.iter().sum::<f64>() / n as f64;
    let deviations: Vec<f64> = data.iter().map(|&x| x - mean).collect();
    let cumulative: Vec<f64> = deviations.iter()
        .scan(0.0, |acc, &x| { *acc += x; Some(*acc) })
        .collect();
    let range = cumulative.iter().cloned().fold(f64::NEG_INFINITY, f64::max)
        - cumulative.iter().cloned().fold(f64::INFINITY, f64::min);
    let std_dev = (data.iter().map(|&x| (x - mean).powi(2)).sum::<f64>() / n as f64).sqrt();
    if std_dev < 1e-10 { return None; }
}

```

```

    Some(range / std_dev)
}

fn ols_slope(x: &f64, y: &f64) -> f64 {
    let n = x.len() as f64;
    let sx: f64 = x.iter().sum();
    let sy: f64 = y.iter().sum();
    let sxy: f64 = x.iter().zip(y.iter()).map(|(a, b)| a * b).sum();
    let sxx: f64 = x.iter().map(|a| a * a).sum();
    (n * sxy - sx * sy) / (n * sxx - sx * sx)
}

```

## 2B. Order Book Imbalance (`src/features/obi.rs`)

```

/// OBI = (bid_vol - ask_vol) / (bid_vol + ask_vol)
/// Result ∈ [-1.0, 1.0] — positif = tekanan beli, negatif = tekanan jual
pub fn compute_obi(ticks: &[Tick]) -> f64 {
    let total_bid: u64 = ticks.iter().map(|t| t.bid_volume as u64).sum();
    let total_ask: u64 = ticks.iter().map(|t| t.ask_volume as u64).sum();
    let total = total_bid + total_ask;
    if total == 0 { return 0.0; }
    (total_bid as f64 - total_ask as f64) / total as f64
}

```

## 2C. Feature Pipeline Orchestrator (`src/features/pipeline.rs`)

### Instruksi untuk AI Coder:

- Terima `&[Tick]` sebagai input (sliding window dari `ChunkContext`).
- Semua 9 fitur dihitung dalam satu pass — hindari multiple iterations.
- Gunakan `WindowConfig` untuk mengontrol window size.
- Output `FeatureRow` per-tick yang valid (setelah warmup terpenuhi).
- **Tidak boleh menggunakan Polars `DataFrame`** di hot path per-tick — Polars digunakan hanya untuk batch analytics atau regime classification, bukan real-time feature computation.

## FASE 3: GP & Bayesian Engine

### 3A. GP Initialization & Operators (`src/gp/operators.rs`)

```

use rand::prelude::*;

/// Grow method: menghasilkan pohon acak dengan random depth
/// Pastikan validasi is_valid() setelah generate
pub fn grow_tree(max_depth: usize, rng: &mut impl Rng) -> AstNode {
    if max_depth == 0 || rng.gen_bool(0.3) {
        AstNode::Leaf(random_terminal(rng))
    } else {
        if rng.gen_bool(0.5) {
            // Unary node
            AstNode::Unary {
                op: random_unary_primitive(rng),
                child: Box::new(grow_tree(max_depth - 1, rng)),
            }
        } else {
            // Binary node
            AstNode::Binary {
                op: random_binary_primitive(rng),
                left: Box::new(grow_tree(max_depth - 1, rng)),
                right: Box::new(grow_tree(max_depth - 1, rng)),
            }
        }
    }
}

/// Subtree Crossover: tukar subtree acak antara dua parent

```

```

/// WAJIB validasi is_valid() pada hasil; retry jika gagal (max 10x)
pub fn crossover(parent_a: &AstNode, parent_b: &AstNode, rng: &mut impl Rng) -> (AstNode, AstNode) {
    // 1. Pilih node acak di parent_a dan parent_b sebagai crossover point
    // 2. Tukar subtrees
    // 3. Validasi bloat constraint
    // 4. Jika invalid, kembalikan parents asli (tournament-style fallback)
    todo!("Implementasikan subtree crossover dengan mutable tree traversal")
}

/// Point Mutation: mutasi satu node secara acak
pub fn mutate(tree: &mut AstNode, rng: &mut impl Rng, mutation_rate: f64) {
    // Traverse tree, untuk setiap node dengan prob mutation_rate:
    // - Jika Leaf: ganti terminal secara acak
    // - Jika Unary/Binary: ganti operator secara acak (maintain arity)
    todo!("Implementasikan point mutation traversal")
}

```

### 3B. PyO3 Bayesian Bridge (`src/gp/bayesian.rs`)

```

use pyo3::prelude::*;
use pyo3::types::PyDict;

/// Wrapper untuk memanggil Optuna TPE dari Rust
pub struct OptunaOptimizer {
    study_name: String,
}

impl OptunaOptimizer {
    /// Minta satu set parameter baru dari Optuna TPE
    /// Rust mengevaluasi parameter tersebut, lalu melaporkan hasilnya kembali
    pub fn suggest_params(
        &self,
        trial_number: u32,
        sl_pips_range: (f64, f64),
    ) -> PyResult<StrategyParams> {
        Python::with_gil(|py| {
            // Import optuna_bridge.py
            let optuna_bridge = py.import("optuna_bridge")?;

            let kwargs = PyDict::new(py);
            kwargs.set_item("study_name", &self.study_name)?;
            kwargs.set_item("trial_number", trial_number)?;
            kwargs.set_item("sl_min", sl_pips_range.0)?;
            kwargs.set_item("sl_max", sl_pips_range.1)?;

            let result = optuna_bridge
                .getattr("suggest_params")?
                .call((), Some(kwargs))?;

            // Extract hasil dari Python dict
            let sl: f64 = result.get_item("sl_pips")?.extract()?;
            let tp: f64 = result.get_item("tp_pips")?.extract()?;
            let threshold: f64 = result.get_item("signal_threshold")?.extract()?;
            let window: usize = result.get_item("feature_window")?.extract()?;

            Ok(StrategyParams { sl_pips: sl, tp_pips: tp, signal_threshold: threshold, feature_window:
window })
        })
    }

    /// Laporkan skor fitness kembali ke Optuna untuk update surrogate model
    pub fn report_result(&self, trial_number: u32, score: f64) -> PyResult<()> {
        Python::with_gil(|py| {
            let optuna_bridge = py.import("optuna_bridge")?;
            let kwargs = PyDict::new(py);
            kwargs.set_item("study_name", &self.study_name)?;
            kwargs.set_item("trial_number", trial_number)?;
            kwargs.set_item("value", score)?;
            optuna_bridge.getattr("report_result")?.call((), Some(kwargs))?;
            Ok(())
        })
    }
}

```

```

    })
  }
}

```

### Script Python (python/optuna\_bridge.py):

```

import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)
_studies: dict = {}

def _get_study(study_name: str) -> optuna.Study:
    if study_name not in _studies:
        _studies[study_name] = optuna.create_study(
            study_name=study_name,
            direction="maximize",
            sampler=optuna.samplers.TPESampler(seed=42)
        )
    return _studies[study_name]

def suggest_params(study_name: str, trial_number: int,
                   sl_min: float = 10.0, sl_max: float = 50.0) -> dict:
    study = _get_study(study_name)
    trial = study.ask()

    sl_pips = trial.suggest_float("sl_pips", sl_min, sl_max)
    tp_pips = trial.suggest_float("tp_pips", sl_pips * 1.5, 150.0)
    signal_threshold = trial.suggest_float("signal_threshold", 0.05, 0.95)
    feature_window = trial.suggest_int("feature_window", 3, 300)

    return {
        "sl_pips": sl_pips, "tp_pips": tp_pips,
        "signal_threshold": signal_threshold, "feature_window": feature_window,
        "_trial_id": trial.number
    }

def report_result(study_name: str, trial_number: int, value: float):
    study = _get_study(study_name)
    study.tell(trial_number, value)

```

### 3C. Diversity Engine (`src/gp/diversity.rs`)

```

/// Tree Edit Distance (Zhang-Shasha simplified) antara dua AST
/// Return: normalized distance  $\in [0.0, 1.0]$ 
/// 0.0 = identik, 1.0 = sangat berbeda
pub fn genotypic_distance(a: &AstNode, b: &AstNode) -> f64 {
    // Implementasikan Zhang-Shasha TED atau approximation:
    // 1. Flatten kedua tree menjadi Vec<NodeLabel>
    // 2. Hitung edit distance (insert/delete/relabel cost = 1)
    // 3. Normalize dengan max(size_a, size_b)
    todo!("Implementasikan Tree Edit Distance")
}

/// Phenotypic similarity berdasarkan trade overlap
/// trades_a dan trades_b = Vec<(entry_time_ns, exit_time_ns, direction)>
/// Return: overlap ratio [0.0, 1.0]
pub fn phenotypic_similarity(
    trades_a: &[(i64, i64, bool)],
    trades_b: &[(i64, i64, bool)],
) -> f64 {
    if trades_a.is_empty() || trades_b.is_empty() { return 0.0; }
    let matching = trades_a.iter().filter(|ta| {
        trades_b.iter().any(|tb| {
            ta.2 == tb.2 && // Arah sama
            overlap_ratio((ta.0, ta.1), (tb.0, tb.1)) > 0.5
        })
    }).count();
    matching as f64 / trades_a.len().max(trades_b.len()) as f64
}

```

```
fn overlap_ratio(a: (i64, i64), b: (i64, i64)) -> f64 {
    let overlap_start = a.0.max(b.0);
    let overlap_end = a.1.min(b.1);
    if overlap_end <= overlap_start { return 0.0; }
    let overlap = (overlap_end - overlap_start) as f64;
    let union = (a.1.max(b.1) - a.0.min(b.0)) as f64;
    overlap / union
}
```

## FASE 4: Backtest & Validation Layer

### 4A. Core Backtest Engine (`src/backtest/engine.rs`)

```
#[derive(Debug, Clone, Copy, PartialEq)]
pub enum Signal { Hold, Buy }

#[derive(Debug, Clone)]
pub struct Position {
    pub entry_time: DateTime<Utc>,
    pub entry_price: f64,
    pub sl_price: f64,
    pub tp_price: f64,
    pub max_hold_ticks: usize,
    pub hold_ticks: usize,
}

/// State machine untuk satu strategi
#[derive(Debug, Clone, Default)]
pub struct StrategyState {
    pub open_position: Option<Position>,
    pub equity: f64,
    pub last_signal: Signal,
}

impl Default for Signal { fn default() -> Self { Signal::Hold } }

/// Core tick-by-tick simulation – dipanggil untuk setiap FeatureRow
/// HARUS pure / deterministik – tidak ada random, tidak ada I/O
pub fn process_tick(
    state: &mut StrategyState,
    feature: &FeatureRow,
    tick: &Tick,
    ast: &AstNode,
    params: &StrategyParams,
    broker_spread_pips: f64, // Base spread untuk transaction cost
    broker_commission: f64, // Komisi tetap per trade
) -> Option<TraderResult> {
    // 1. Evaluasi AST terhadap feature → raw_signal f64
    let raw_signal = ast.evaluate(feature);

    // 2. Compute signal berdasarkan threshold
    let current_signal = if raw_signal > params.signal_threshold {
        Signal::Buy
    } else {
        Signal::Hold
    };

    // 3. Stateful Signal Transition: HANYA entry pada HOLD → BUY
    let should_enter = state.last_signal == Signal::Hold && current_signal == Signal::Buy;
    state.last_signal = current_signal;

    // 4. Kelola posisi terbuka
    if let Some(ref mut pos) = state.open_position {
        pos.hold_ticks += 1;

        let exit_reason = if tick.bid <= pos.sl_price {
            Some("SL_HIT")
        } else if tick.ask >= pos.tp_price {
```



```

        Some("TP_HIT")
    } else if pos.hold_ticks >= pos.max_hold_ticks {
        Some("TIMEOUT")
    } else {
        None
    }
};

if let Some(reason) = exit_reason {
    let gross_pips = (tick.bid - pos.entry_price) / 0.0001; // Untuk 5-digit pair
    let net_pips = gross_pips - broker_spread_pips - broker_commission;
    state.equity += net_pips;
    state.open_position = None;
    return Some(TradeResult {
        entry_time: pos.entry_time,
        exit_time: tick.timestamp,
        gross_pips,
        net_pips,
        exit_reason: reason.to_string(),
    });
}
}

// 5. Buka posisi baru jika tidak ada posisi dan sinyal valid
if should_enter && state.open_position.is_none() {
    let entry = tick.ask; // Entry pada ask price
    state.open_position = Some(Position {
        entry_time: tick.timestamp,
        entry_price: entry,
        sl_price: entry - params.sl_pips * 0.0001,
        tp_price: entry + params.tp_pips * 0.0001,
        max_hold_ticks: params.feature_window * 10,
        hold_ticks: 0,
    });
}

None
}

```

#### 4B. Hard Filter (`src/backtest/filter.rs`)

```

/// Eliminasi instan – dipanggil setelah backtest satu window selesai
pub fn passes_hard_filters(metrics: &WindowMetrics) -> bool {
    // Frekuensi trade: 10-20 per bulan
    if metrics.trade_count < 10 || metrics.trade_count > 20 { return false; }
    // Risk per trade: 10-50 pips
    if metrics.avg_sl_pips < 10.0 || metrics.avg_sl_pips > 50.0 { return false; }
    // Max drawdown per bulan
    if metrics.max_drawdown_pips > 500.0 { return false; }
    // Harus ada minimal 1 loss (Profit Factor bukan infinity)
    if metrics.loss_count == 0 { return false; }
    true
}

/// OOS Degradation Rule: bunuh strategi jika OOS PF < 1.5 di ≥80% windows
pub fn passes_degradation_filter(oos_results: &[f64]) -> bool {
    if oos_results.is_empty() { return false; }
    let failed = oos_results.iter().filter(|&pf| pf < 1.5).count();
    let fail_ratio = failed as f64 / oos_results.len() as f64;
    fail_ratio < 0.80 // Lulus jika fail rate di bawah 80%
}

```

#### 4C. Walk-Forward Windows (`src/backtest/walk\_forward.rs`)

```

#[derive(Debug, Clone)]
pub struct WalkForwardWindow {
    pub window_id: usize,
    pub is_months: Vec<(i32, u32)>, // (year, month) pairs for In-Sample
    pub oos_months: Vec<(i32, u32)>, // (year, month) pairs for Out-of-Sample
}

```

```

/// Generate matriks rolling windows
/// is_size: jumlah bulan In-Sample (contoh: 6)
/// oos_size: jumlah bulan Out-of-Sample (contoh: 2)
/// Contoh output: [(IS: 1-6, OOS: 7-8), (IS: 3-8, OOS: 9-10), ...]
pub fn generate_windows(
  start_year: i32,
  start_month: u32,
  total_months: usize,
  is_size: usize,
  oos_size: usize,
  step: usize, // Berapa bulan setiap shift (default: 2 = sama dengan oos_size)
) -> Vec<WalkForwardWindow> {
  let mut windows = Vec::new();
  let mut offset = 0;
  let mut window_id = 0;

  while offset + is_size + oos_size <= total_months {
    // Hitung is_months dan oos_months dari (start + offset)
    // ... implementasi kalkulasi bulan
    offset += step;
    window_id += 1;
  }

  windows
}

```

#### 4D. Regime Classifier (`src/backtest/regime.rs`)

```

pub fn classify_regime(candles: &[Candle]) -> RegimeLabel {
  let closes: Vec<f64> = candles.iter().map(|c| c.close).collect();
  let h = compute_hurst(&closes);
  let atr = compute_atr(candles);
  let atr_threshold = compute_atr_median_threshold(candles); // Baseline ATR

  let is_trending = h > 0.55;
  let is_ranging = h < 0.45;
  let is_high_vol = atr > atr_threshold;

  match (is_high_vol, is_trending, is_ranging) {
    (true, true, _) => RegimeLabel::HighVolTrending,
    (true, _, true) => RegimeLabel::HighVolRanging,
    (false, true, _) => RegimeLabel::LowVolTrending,
    (false, _, true) => RegimeLabel::LowVolRanging,
    _ => RegimeLabel::RandomNoise,
  }
}

fn compute_atr(candles: &[Candle]) -> f64 {
  if candles.len() < 2 { return 0.0; }
  let trs: Vec<f64> = candles.windows(2).map(|w| {
    let curr = &w[1];
    let prev = &w[0];
    let tr1 = curr.high - curr.low;
    let tr2 = (curr.high - prev.close).abs();
    let tr3 = (curr.low - prev.close).abs();
    tr1.max(tr2).max(tr3)
  }).collect();
  trs.iter().sum::<f64>() / trs.len() as f64
}

```

## FASE 5: Dashboard & Observability

### 5A. Axum Server Setup (`src/dashboard/server.rs`)

```

use axum::{Router, routing::get};
use tower_http::cors::CorsLayer;
use std::sync::Arc;

```

```
pub async fn run_dashboard(telemetry: SharedTelemetry, port: u16) {
    let app = Router::new()
        .route("/api/evolution/overview", get(api::evolution::get_overview))
        .route("/api/strategy/:id/deep-dive", get(api::strategy::get_deep_dive))
        .route("/api/regime/heatmap", get(api::regime::get_heatmap))
        .route("/api/system/telemetry", get(api::system::get_telemetry))
        // WebSocket endpoint untuk live streaming metrics
        .route("/ws/live", get(ws_live_handler))
        .layer(CorsLayer::permissive())
        .with_state(telemetry);

    let listener = tokio::net::TcpListener::bind(format!("0.0.0.0:{port}"))
        .await.unwrap();
    axum::serve(listener, app).await.unwrap();
}
```

## 5B. Main Entry Point (`src/main.rs`)

```
#[tokio::main]
async fn main() -> anyhow::Result<()> {
    // 1. Init tracing subscriber
    tracing_subscriber::fmt()
        .with_env_filter("esse=debug,tower_http=info")
        .json()
        .init();

    // 2. Load config
    let config = Config::from_file("config/default.toml")?;

    // 3. Init shared state
    let telemetry: SharedTelemetry = Arc::new(parking_lot::RwLock::new(TelemetryState::default()));

    // 4. Init DB connections
    let ch_client = ClickHouseDataSource::new(&config.clickhouse_url, "geonera");
    let pg_pool = sqlx::PgPool::connect(&config.postgres_url).await?;
    sqlx::migrate!("./migrations").run(&pg_pool).await?;

    // 5. Spawn Dashboard sebagai task terpisah – NOL GESEKAN terhadap engine
    let telemetry_dashboard = Arc::clone(&telemetry);
    tokio::spawn(async move {
        dashboard::server::run_dashboard(telemetry_dashboard, config.dashboard_port).await;
    });

    // 6. Run Evolution Engine (main loop – blocking sampai done)
    evolution::orchestrator::run(config, ch_client, pg_pool, telemetry).await?;

    Ok(())
}
```

# BAGIAN 5: Prompting Rules & Constraints untuk AI Coder

## 5.1 Memory & Concurrency Safety

- RULE-MEM-01: DILARANG memuat seluruh dataset tick ke Vec<Tick> sekaligus.  
Selalu gunakan streaming iterator atau tokio\_stream::Stream.
- RULE-MEM-02: SharedTelemetry menggunakan parking\_lot::RwLock.  
Write lock HANYA boleh dipegang maksimal 5ms.  
Jangan pernah melakukan I/O di dalam lock.
- RULE-MEM-03: Arc<T> untuk shared ownership. Box<T> untuk single ownership (AST nodes).  
Hindari Rc<T> dan RefCell<T> – sistem ini multi-threaded.
- RULE-MEM-04: Semua tokio::spawn harus menerima argumen Send + 'static.  
Jika struct tidak Send, gunakan spawn\_blocking atau ubah struktur.

RULE-CON-01: Pisahkan CPU-bound work dari async runtime.  
Gunakan: `tokio::task::spawn_blocking(|| rayon::scope(|s| { ... })))`  
JANGAN jalankan loop evaluasi AST langsung di dalam async fn.

RULE-CON-02: Channel messaging antara Evolution Engine dan Dashboard:  
Gunakan `tokio::sync::broadcast<TelemetrySnapshot>` untuk fan-out.  
Capacity broadcast channel: 128 items.

RULE-CON-03: Populasi GP (`Vec<Individual>`) dibungkus dalam `Arc<RwLock<Population>>`.  
Selection dan breeding memerlukan read lock.  
Penulisan generasi baru memerlukan write lock – minimisasi durasi.

## 5.2 Performance & Correctness

RULE-PERF-01: Inner loop evaluasi AST (di dalam backtest per-tick) harus BEBAS dari heap allocation. Hindari `Vec::new()`, `String::from()`, `Box::new()` di dalam loop hot path.

RULE-PERF-02: Aktifkan SIMD secara implisit dengan compiler flags:  
`RUSTFLAGS="-C target-cpu=native" cargo build --release`  
Tulis kode numerik dalam bentuk yang dapat di-auto-vectorize  
(loop sederhana, tidak ada branching kompleks di dalam loop body).

RULE-PERF-03: Evaluasi populasi besar menggunakan `rayon::par_iter()`:  
`population.par_iter_mut().for_each(|individual| evaluate(individual, &data));`

RULE-PERF-04: ClickHouse query gunakan kompresi LZ4 (sudah dikonfigurasi di `Cargo.toml`).  
Fetch column-oriented, bukan row-by-row.

RULE-PERF-05: PostgreSQL batch insert menggunakan UNNEST atau COPY untuk `trade_logs` – jangan INSERT satu per satu dalam loop.

RULE-CORRECT-01: Protected Division WAJIB – periksa denominator  $< 1e-10$  sebelum bagi.  
Periksa domain Log ( $v > 0$ ) dan Sqrt ( $v \geq 0$ ).  
NaN dalam evaluasi AST = strategi langsung di-eliminasi.

RULE-CORRECT-02: Walk-Forward IS Blinding – Optuna study HANYA boleh menerima feedback dari window In-Sample. Skor OOS TIDAK boleh masuk ke Optuna selama tuning. Ini constraint kritis anti-leakage.

RULE-CORRECT-03: Timestamp selalu UTC. Gunakan `chrono::DateTime<Utc>` di semua struct.  
DILARANG menggunakan `DateTime<Local>` atau `NaiveDateTime` untuk data pasar.

RULE-CORRECT-04: Hurst Exponent membutuhkan minimal 20 data points.  
Return 0.5 (Random Noise) jika data tidak cukup.

## 5.3 GP & Evolutionary Algorithm

RULE-GP-01: Setiap operasi mutasi dan crossover WAJIB diikuti pemanggilan `is_valid()` pada pohon hasil. Jika invalid (bloat violation), kembalikan parent asli. DILARANG menghasilkan pohon invalid.

RULE-GP-02: Konstanta `Terminal::Constant(f64)` harus di-clip ke range `[-100.0, 100.0]` untuk mencegah eksplosif nilai numerik dalam AST.

RULE-GP-03: Phenotypic distance threshold 85% similarity:  
`if phenotypic_similarity(...) > 0.85 {  
 apply_destructive_mutation(individual, rng);  
 // ATAU: individual.fitness -= CROWDING_PENALTY;  
}`

RULE-GP-04: Diversity check dilakukan SEBELUM individu baru masuk populasi Elite.  
Jangan check setelah – ini terlambat.

RULE-GP-05: Bayesian FFI ke Python via PyO3 HARUS dijalankan dalam `tokio::task::spawn_blocking` karena Python GIL bersifat blocking.  
DILARANG memanggil Python dari dalam async fn secara langsung.

## 5.4 Architecture & Code Quality

- RULE-ARCH-01: Setiap modul HARUS expose interface melalui trait, bukan struct langsung.  
Contoh: `trait DataSource { async fn stream_ticks(...) -> ...; }`  
Ini memungkinkan unit testing dengan mock implementation.
- RULE-ARCH-02: Custom error enum `EsseError` via `thiserror`.  
Jangan menggunakan `unwrap()` atau `expect()` di production code.  
Gunakan `?` operator dan propagasi error ke atas.
- RULE-ARCH-03: DILARANG menggunakan `static mut` atau `lazy_static` untuk mutable state.  
Gunakan `Arc<RwLock<T>>` atau `tokio::sync::Mutex<T>`.
- RULE-ARCH-04: Semua konfigurasi runtime (DB URLs, hyperparameter) dari `config/default.toml`.  
DILARANG hardcode nilai apapun yang mungkin berubah di source code.
- RULE-ARCH-05: Feature computation HARUS idempotent – input yang sama menghasilkan output yang sama. Ini wajib untuk reproducibility penelitian.
- RULE-TEST-01: Setiap fungsi fitur (`obi`, `hurst`, `dll`) HARUS memiliki unit test dengan data sintetis yang nilai ekspektasinya diketahui secara analitik.
- RULE-TEST-02: Backtest engine HARUS deterministik – seed RNG eksplisit, tidak ada concurrency non-deterministik di dalam simulation loop.

## 5.5 GPU Acceleration (Opsional / Feature-gated)

- RULE-GPU-01: Semua kode GPU berada di bawah feature flag `#[cfg(feature = "gpu-cuda")]`.  
Sistem harus berfungsi penuh dengan feature `"cpu"` default.
- RULE-GPU-02: `AST` → `PTX` compilation via `cudarc/nvrtc`.  
Representasi kernel: setiap node `AST` = satu GPU thread instruction.  
Batch: evaluasi  $N \text{ individu} \times M \text{ tick}$  secara paralel di VRAM.
- RULE-GPU-03: Transfer data CPU↔GPU diminimisasi.  
Upload feature matrix SEKALI per generasi ke VRAM.  
Download hanya fitness scores (scalar per individu).
- RULE-GPU-04: WGPU sebagai fallback cross-platform (tanpa CUDA).  
Gunakan compute shaders WGSL untuk evaluasi `AST`.

## Appendix: Urutan Implementasi yang Disarankan

Ikuti urutan ini secara ketat untuk menghindari dependency hell:

*Sprint | Komponen | Output Terverifikasi*

- 1 | ``error.rs``, ``config.rs``, tipe dasar (``Tick``, ``Candle``, ``Instrument``) | Unit test compile & serialize
- 2 | ``data/clickhouse.rs`` (streaming) | Dapat stream 1 bulan XAUUSD tick
- 3 | ``data/postgres.rs`` + migrations | CRUD strategy\_dna berhasil
- 4 | ``features/`` semua 9 fitur | Unit test nilai masing-masing fitur
- 5 | ``gp/ast.rs`` + evaluator | Evaluate tree sederhana tanpa panic
- 6 | ``gp/population.rs`` + operators | Generate 1000 valid trees
- 7 | ``gp/bayesian.rs`` + Python bridge | Optuna suggest + report round-trip
- 8 | ``backtest/engine.rs`` | Backtest 1 strategi 1 bulan data
- 9 | ``backtest/walk_forward.rs`` + filters | Full pipeline 1 individu
- 10 | ``gp/diversity.rs`` | Diversity check funsi
- 11 | ``evolution/orchestrator.rs`` | 1 generasi penuh end-to-end

12 | ``dashboard/server.rs`` + API | Dashboard live saat engine jalan  
13 | Noise injection + regime + degradation | Filter pipeline lengkap  
14 | Top-N selection + JSON export | Golden Strategy Set ter-export  
15 | GPU path (opsional) | Performance benchmark CPU vs GPU

---

*ESSE v1.0 Technical Blueprint — Dokumen ini adalah panduan implementasi untuk AI Coding Agent. Semua tipe, konstanta, dan formula bersifat presisi dan binding. Deviasi dari spesifikasi ini harus didokumentasikan dengan justifikasi teknis yang jelas.*